

Recuperación Examen Práctico

Programación Orientada a Objetos

RA2 – RA4- RA7

Resultados de Aprendizaje y Criterios de Evaluación aplicados

RA2	Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.
a	Se han identificado los fundamentos de la programación orientada a Objetos.
b	Se han escrito programas simples.
c	Se han instanciado objetos a partir de clases predefinidas.
d	Se han utilizado métodos y propiedades de los objetos.
e	Se han escrito llamadas a métodos estáticos.
f	Se han utilizado parámetros en la llamada a métodos.
g	Se han incorporado y utilizado librerías de objetos.
h	Se han utilizado constructores.
i	Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples.
RA4	Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.
a	Se ha reconocido la sintaxis, estructura y componentes típicos de una Clase.
b	Se han definido clases.
c	Se han definido propiedades y métodos.
d	Se han creado constructores.
e	Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.
f	Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros.
g	Se han definido y utilizado clases heredadas
h	Se han creado y utilizado métodos estáticos.
i	Se han creado y utilizado conjuntos y librerías de clases
RA7	Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.
a	Se han identificado los conceptos de herencia, superclase y subclase.
b	Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos.
c	Se ha reconocido la incidencia de los constructores en la herencia.
d	Se han creado clases heredadas que sobrescriben la implementación de métodos de la superclase.

e	Se han diseñado y aplicado jerarquías de clases.
f	Se han probado y depurado las jerarquías de clases.
g	Se han realizado programas que implementen y utilicen jerarquías de clases.
h	Se ha comentado y documentado el código.
i	Se han identificado y evaluado los escenarios de uso de interfaces.
j	Se han identificado y evaluado los escenarios de utilización de la herencia y la composición.

Todos los RA y CE se aplican en la tarea. En cada apartado se indica la ponderación a aplicar en base a la calificación definitiva de la prueba. Dicha calificación será la misma para todos los RAs aplicados, ya que están estrechamente relacionados con los fundamentos introductorios y avanzados de la Programación Orientada a Objetos.

Enunciado

1. Crea un proyecto, en IntelliJ, llamado "**Rec_RA247_Nombre_Apellidos**" (Ej: *Rec_RA247_Miguel_Ángel_Sánchez_Benito*).
2. Dentro crea la siguiente estructura de paquetes, clases e interfaces:
 - a. src
 - i. com.beer
 - cervezas
 - Cerveza (clase abstracta)
 - Lager (clase)
 - IPA (clase)
 - main
 - Main (clase principal)
 - utils
 - lArtesanal (interfaz)
 - Cerveceria (clase)
 - CervezaUtils (clase)

3. A continuación, se detalla el contenido a desarrollar en cada una de las clases/interfaces, junto a los **puntos** que vale cada apartado:

a. **[2 puntos] Clase Cerveza:**

i. Crea una clase abstracta `Cerveza` que sirva como base para diferentes tipos de cervezas. Esta clase debe tener los siguientes atributos:

- `nombre`: Nombre de la cerveza (String).
- `graduacionAlcoholica`: Graduación alcohólica (double).
- Un atributo estático `totalCervezas` que llevará el control del número total de cervezas creadas.

ii. La clase debe tener los siguientes métodos:

- Un constructor que reciba el nombre y la graduación alcohólica. Aquí hay que incrementar el número total de cervezas creadas.
- Métodos abstractos `servir()` y `maridar()`, que serán implementados por las subclases.
- Un método estático `getTotalCervezas()` que devuelva la cantidad total de cervezas creadas.
- Métodos `getNombre()` y `getGraduacionAlcoholica()` para acceder a los atributos.

b. **[1 punto] Interfaz IArtesanal:**

i. Crea una interfaz llamada `IArtesanal` que permita a ciertas cervezas indicar si son artesanales. Esta interfaz debe tener los siguientes métodos:

- `certificarArtesanal()`: Devuelve un mensaje indicando que la cerveza es artesanal.
- `origen()`: Devuelve el país o región de origen de la cerveza.

c. **[2 puntos] Clase IPA:**

i. Crea la clase `IPA`, que extiende `Cerveza` e implementa `IArtesanal`. Representará cervezas tipo India Pale Ale.

ii. Atributos:

- `amargor`: Nivel de amargor en IBU (Unidad Internacional de Amargor) (int).
- `origen`: Lugar de origen (String).

iii. Métodos:

- Constructor: Recibe el nombre, graduación alcohólica, amargor y origen.

- Implementa los métodos `servir()` y `maridar()`, que mostrarán un mensaje con cómo servir y con qué alimentos combinar la cerveza.
- Implementa `certificarArtesanal()` y `origen()`.
- Métodos `getAmargor()` y `setAmargor()` para acceder y modificar el nivel de amargor.

d. **[2 puntos] Clase Lager:**

- i. Crea la clase `Lager`, que también extiende `Cerveza`. Representará cervezas tipo Lager.
- ii. Atributos:
 - `fermentacionBaja`: Indica si es de fermentación baja (boolean).
- iii. Métodos:
 - Constructor: Recibe el nombre, graduación alcohólica y si es de fermentación baja.
 - Implementa los métodos `servir()` y `maridar()`, que mostrarán un mensaje con instrucciones de servicio y maridaje.
 - Métodos `isFermentacionBaja()` y `setFermentacionBaja()` para acceder y modificar la propiedad.

e. **[1 punto] Clase Cerveceria:**

- i. Crea una clase `Cerveceria` que representará un conjunto de cervezas.
- ii. Atributos:
 - `nombre`: Nombre de la cervecería (String).
 - `catalogo`: Una lista de objetos `Cerveza` que forman el catálogo de la cervecería. (ArrayList de `Cerveza`).
- iii. Métodos:
 - Constructor: Recibe el nombre de la cervecería e inicializa el ArrayList.
 - Métodos `agregarCerveza(Cerveza cerveza)` y `eliminarCerveza(Cerveza cerveza)`.
 - Método `mostrarCatalogo()` que imprime la lista de cervezas disponibles.

- f. **[1 punto]** Clase `CervezaUtils`:
- i. Crea una clase `CervezaUtils` que contendrá métodos estáticos para interactuar con las cervezas:
- `mostrarDetallesCerveza(Cerveza cerveza)`: Muestra por pantalla el nombre y la graduación alcohólica de la cerveza.
 - `compararAmargor(IPA ipa1, IPA ipa2)`: Compara dos cervezas IPA y muestra cuál es más amarga.
 - `recomendarMaridaje(Cerveza cerveza)`: Sugiere un maridaje basado en el tipo de cerveza.
- g. **[1 punto]** Clase y método `main()`:
- i. Crea un programa en el método `main()` que realice las siguientes acciones:
- Crea al menos dos cervezas de tipo `IPA` y una de tipo `Lager`.
 - Crea una `Cerveceria` y agrega las cervezas creadas.
 - Utiliza la clase `CervezaUtils` para mostrar los detalles de las cervezas y comparar dos IPAs.
 - Imprime el número total de cervezas creadas utilizando el método estático `getTotalCervezas()` de la clase `Cerveza`.

- **PUNTOS ADICIONALES (Según RAs a recuperar)**

- **RA2**

- Añade, en el `main`:
 - Cervezas de tipo `BlackLager` a la cervecería ya existente. Al menos, crea dos.
 - Imprime también sus datos, utilizando la clase `CervezaUtils` (igual que con las anteriores).
 - Imprime el total de cervezas creadas, donde deben estar sumadas las nuevas `BlackLager`.

- **RA4**

- Crea la clase `BlackLager`, que también extiende `Cerveza`. Representará cervezas tipo Black Lager (oscura).
 - Atributos:
 - `sabor`: Indica el tipo de sabor (String). Ejemplos: “chocolate”, “café”, “nueces”, “pan tostado”...
 - Métodos:
 - Constructor: Recibe el nombre, graduación alcohólica y sabor.
 - Implementa los métodos `servir()` y `maridar()`, que mostrarán un mensaje con instrucciones de servicio y maridaje.
 - Métodos `getSabor()` y `setSabor()` para acceder y modificar la propiedad.

- **RA7**

- Nada más

- **Normas y Entrega:**
 - La aplicación debe compilar. No se considerará aprobado si no compila.
 - La aplicación debe ejecutar razonablemente de acuerdo a los requisitos solicitados.
 - Comprime tu directorio en un archivo ZIP con exactamente la misma nomenclatura, pero extensión ZIP:
 - **Rec_RA247_Nombre_Apellidos.zip**
 - Ejemplo: "Rec_RA247_Miguel_Angel_Sanchez_Benito.zip".
 - Sube tu archivo ZIP a Aules
 - **El incumplimiento de alguna de estas normas será penalizado en la calificación.**

- **Rúbricas → Ver calificación en Aules**